Multistream-Select 프로토콜 동작 방식 분석

개요

Multistream-select는 libp2p에서 사용하는 프로토콜 네고시에이션 프로토콜입니다. 두 피어가 연결을 설정한 후, 어떤 애플리케이션 레벨 프로토콜을 사용할지 협상하는 데 사용됩니다. 이 문서는 Rust libp2p 구현체에서 multistream-select 프로토콜의 동작 방식을 상세히 분석합니다.

프로토콜 역할 및 책임

기본 개념

Multistream-select 프로토콜에서는 두 가지 역할이 있습니다:

1. **Dialer (개시자)**: 프로토콜 네고시에이션을 주도하는 역할
2. **Listener (응답자)**: 개시자의 요청에 응답하는 역할

프로토콜 버전

현재 지원되는 버전:

- **V1**: 표준 multistream-select 1.0.0 프로토콜
- **V1Lazy**: 0-RTT 최적화를 지원하는 V1의 변형

프로토콜 메시지 구조

메시지 타입

```rust
pub(crate) enum Message {
/// 프로토콜 헤더 (multistream-select 버전 식별)
Header(HeaderLine),
/// 프로토콜 제안 또는 확인
Protocol(Protocol),
/// 지원 프로토콜 목록 요청
ListProtocols,
/// 지원 프로토콜 목록 응답
Protocols(Vec<Protocol>),
/// 프로토콜 거부 응답
NotAvailable,
}
```

프로토콜 상수

```rust
/// multistream-select 1.0.0 헤더
const MSG_MULTISTREAM_1_0: &[u8] = b"/multistream/1.0.0\n";
/// 프로토콜 거부 메시지
const MSG_PROTOCOL_NA: &[u8] = b"na\n";
/// 프로토콜 목록 요청 메시지
const MSG_LS: &[u8] = b"ls\n";
```

Dialer (개시자) 동작 방식

상태 머신

Dialer는 다음과 같은 상태를 거쳐 프로토콜 네고시에이션을 수행합니다:

```
enum State<R, N> {
SendHeader { io: MessageIO<R> }, // 헤더 전송
SendProtocol { io: MessageIO<R>, protocol: N }, // 프로토콜 제안
FlushProtocol { io: MessageIO<R>, protocol: N }, // 데이터 플러시
AwaitProtocol { io: MessageIO<R>, protocol: N }, // 응답 대기
Done, // 완료
}
```

네고시에이션 과정

1. **헤더 전송**: `/multistream/1.0.0\n` 헤더를 전송하여 multistream-select 프로토콜 사용을 알림
2. **프로토콜 제안**: 지원하는 프로토콜 중 하나를 제안
3. **응답 대기**: Listener의 응답을 기다림

- 프로토콜 확인: 같은 프로토콜명이 돌아오면 성공
- 프로토콜 거부: `na\n` 이 돌아오면 다음 프로토콜 시도

4. **반복**: 모든 프로토콜이 거부될 때까지 또는 하나가 수락될 때까지 반복

V1Lazy 최적화

V1Lazy 모드에서는 Dialer가 단일 프로토콜만 지원하는 경우 0-RTT 네고시에이션을 수행합니다:

```
Version::V1Lazy => {

// 유일한 프로토콜에 대해 낙관적으로 정착

let hl = HeaderLine::from(Version::V1Lazy);

let io = Negotiated::expecting(io.into_reader(), p, Some(hl));

return Poll::Ready(Ok((protocol, io)));

}
```

Listener (응답자) 동작 방식

상태 머신

```
enum State<R, N> {

RecvHeader { io: MessageIO<R> }, // 헤더 수신

SendHeader { io: MessageIO<R> }, // 헤더 응답

RecvMessage { io: MessageIO<R> }, // 메시지 수신

SendMessage { io: MessageIO<R>, message: Message, protocol: Option<N> }, // 메시지

Flush { io: MessageIO<R>, protocol: Option<N> }, // 플러시

Done,

}
```

네고시에이션 과정

1. **헤더 수신**: Dialer로부터 `/multistream/1.0.0\n` 헤더 수신

2. **헤더 응답**: 같은 헤더로 응답하여 프로토콜 지원 확인

3. **메시지 처리**:

- **프로토콜 제안 수신**: 지원하는 프로토콜인지 확인
- **프로토콜 목록 요청**: `ls\n` 메시지에 대해 지원 프로토콜 목록 응답

4. **응답 전송**:

- 지원하는 프로토콜: 같은 프로토콜명으로 확인 응답
- 지원하지 않는 프로토콜: `na\n` 으로 거부 응답

프로토콜 매칭 로직

```
let protocol = this.protocols.iter().find_map(|(name, proto)| {
if &p == proto {
Some(name.clone())
} else {
None
}
});

let message = if protocol.is_some() {
Message::Protocol(p.clone()) // 확인
} else {
Message::NotAvailable // 거부
};
```

메시지 인코딩/디코딩

메시지 인코딩

각 메시지는 길이 접두사가 붙은 형태로 인코딩됩니다:

```
fn encode(&self, dest: &mut BytesMut) {
match  self {
Message::Header(HeaderLine::V1) => {
dest.put(MSG_MULTISTREAM_1_0);
}
Message::Protocol(p) => {
dest.put(p.0.as_ref());
dest.put_u8(b'\n');
}
Message::NotAvailable => {
dest.put(MSG_PROTOCOL_NA);
}
// ... 기타 메시지 타입
}
}
```

메시지 디코딩

수신된 바이트 스트림을 파싱하여 메시지 타입을 결정합니다:

```
fn decode(mut msg: Bytes) -> Result<Message, ProtocolError> {
if msg == MSG_MULTISTREAM_1_0 {
return Ok(Message::Header(HeaderLine::V1));
}
if msg == MSG_PROTOCOL_NA {
return Ok(Message::NotAvailable);
}
// 프로토콜명 파싱: '/'로 시작하고 '\n'으로 끝남
if msg.first() == Some(&b'/') && msg.last() == Some(&b'\n') {
let p = Protocol::try_from(msg.split_to(msg.len() - 1))?;
return Ok(Message::Protocol(p));
}
// ... 기타 메시지 파싱 로직
}
```

Negotiated 스트림

네고시에이션 완료 후 상태

프로토콜 네고시에이션이 완료되면 `Negotiated<TInner>` 스트림이 반환됩니다:

```
pub struct Negotiated<TInner> {
state: State<TInner>,
}
```

```
enum State<R> {
/// 아직 프로토콜 확인을 기다리는 상태 (V1Lazy용)
Expecting {
io: MessageReader<R>,
protocol: Protocol,
header: Option<HeaderLine>,
},
/// 네고시에이션 완료, 애플리케이션 데이터 통신 가능
Completed {
io: R,
},
}
```

V1Lazy에서의 지연 확인

V1Lazy 모드에서는 네고시에이션이 완료되기 전에 애플리케이션 데이터 전송을 시작할 수 있습니다:

```
impl<TInner> AsyncRead for Negotiated<TInner> {
fn poll_read(&mut  self, cx: &mut Context<'_>, buf: &mut [u8]) -> Poll<Result<us
loop {
if  let StateProj::Completed { io } = self.project().state.project() {
return io.poll_read(cx, buf);
}
```

```
// 네고시에이션 완료까지 대기
match  self.poll(cx) {
Poll::Ready(Ok(())) => {}
Poll::Pending => return Poll::Pending,
Poll::Ready(Err(err)) => return Poll::Ready(Err(From::from(err))),
}
}
}
}
```

실제 사용 예제

Swarm에서의 사용

libp2p-swarm에서는 서브스트림 업그레이드 시 multistream-select를 사용합니다:

```
// Dialer (outbound) 연결
let (info, stream) = multistream_select::dialer_select_proto(
substream,
protocols,
effective_version,
)
.await
.map_err(to_stream_upgrade_error)?;
```

```
// Listener (inbound) 연결
let (info, stream) = multistream_select::listener_select_proto(substream, protoc
.await
.map_err(to_stream_upgrade_error)?;
```

프로토콜 정보 제공

각 프로토콜은 `ProtocolInfo` 를 통해 자신의 식별자를 제공합니다:

```
// Kademlia DHT 예제
pub(crate) const DEFAULT_PROTO_NAME: StreamProtocol = StreamProtocol::new("/ipfs

// AutoNAT 예제
pub  const DEFAULT_PROTOCOL_NAME: StreamProtocol = StreamProtocol::new("/libp2p/
```

오류 처리

프로토콜 오류 타입

```
pub  enum ProtocolError {
IoError(io::Error), // I/O 오류
InvalidMessage, // 잘못된 메시지
InvalidProtocol, // 잘못된 프로토콜명
```

```
TooManyProtocols, // 너무 많은 프로토콜
}

pub  enum NegotiationError {
ProtocolError(ProtocolError), // 프로토콜 오류
Failed, // 네고시에이션 실패
}
```

오류 상황 처리

1. **네트워크 오류**: 연결 끊김, 타임아웃 등
2. **프로토콜 오류**: 잘못된 메시지 형식, 프로토콜 위반
3. **네고시에이션 실패**: 공통 프로토콜 없음

성능 최적화

V1Lazy의 0-RTT 최적화

- 단일 프로토콜 지원 시 즉시 애플리케이션 데이터 전송 가능
- 네고시에이션 메시지와 애플리케이션 데이터를 함께 전송
- 네트워크 라운드트립 감소

메시지 배치 처리

- 헤더와 첫 번째 프로토콜 제안을 함께 전송
- 플러시 최적화로 네트워크 효율성 향상

보안 고려사항

프로토콜 검증

- 프로토콜명은 반드시 '/'로 시작해야 함
- 최대 프로토콜 수 제한 (MAX_PROTOCOLS = 1000)
- UTF-8 유효성 검사

DoS 공격 방어

- 메시지 크기 제한
- 프로토콜 수 제한
- 타임아웃 설정

이 분석을 통해 multistream-select 프로토콜이 어떻게 libp2p에서 프로토콜 네고시에이션을 수행하는지, 그리고 V1과 V1Lazy 버전의 차이점과 최적화 방법을 이해할 수 있습니다.